CS4475 Fall 2026
Gael Sustaita
gsustaita4@miners.utep.edu

Lab Report
Submitted on 10/29/2026

PA3: Parallel MergeSort

Repository: https://github.com/gael68/pa3-parallel-mergesort, branch `main`. The first commit is the provided `parallelMergesort.c` unchanged, so `git diff` against it shows exactly what I added. This report is committed as `PA3 Report.pdf`, and every raw Bridges-2 output file is in `results/`.

Platform: PSC Bridges-2, RM-shared partition. Each node has two AMD EPYC 7742 64-core processors (128 cores, 2 NUMA nodes, 256 GB RAM). Compiler: `gcc (GCC) 8.5.0`. I compiled exactly as the handout shows, `gcc -o mergesortN parallelMergesort.c -lpthread` (no optimization flags), and used one executable per thread count.

How the runs were done: instead of `interact`, I used a batch script (`run.sbatch`) that requests `--ntasks-per-node=16` on RM-shared, which is the same 16-core allocation the interact command in the handout gets. The script builds `mergesort1`, `mergesort2`, `mergesort4`, `mergesort8` and `mergesort16` by setting THREAD_LEVEL to 0–4 (threads = 2^THREAD_LEVEL), then runs each one **5 times** on 10,000,000 elements. Job 50358707 ran on node r195. A second job (50358714, node r358, 64 cores) ran the extra experiments described at the end of Task 2, and a third (50358766, node r227, 16 cores) collected the evidence at the end of Task 1 and the input-size sweep.

THREAD_LEVEL note: the handout says the default is 2^3, but the copy of `parallelMergesort.c` in the shared directory has `#define THREAD_LEVEL 1` (2 threads). It does not matter here because every build sets it explicitly; the script checks that the substitution happened and stops if it did not.

Validating the output: every program run prints its sequential time, parallel time, qsort time and my new merge timers, and the program itself checks that each result is sorted (`isSorted`). I wrote `analysis/parse.py` to read the raw output and refuse to produce any numbers unless every run has all of its timing lines, no run prints `Oops` or ERROR, all times are positive, and each configuration has the expected number of runs. All 100 runs passed (plus 6 profiling runs). I also tested the validator by feeding it a deliberately damaged copy of the output (a deleted line, an injected `Oops`, a missing run); it reported all of them and exited with an error.

Task 1. (10 pts) Speedup with 1, 2, 4, 8 and 16 cores

Each program run times the sequential mergesort and then the parallel mergesort on a copy of the same random list. Speedup is the mean sequential time divided by the mean parallel time for that configuration (5 runs each). Efficiency is speedup / threads. Across all 25 runs the sequential mergesort averaged 2.519 s, so 10,000,000 elements gives a few seconds of sequential time as the handout asks.

Table 1. Execution times on Bridges-2 (N = 10,000,000, 5 runs per configuration, job 50358707). Level = THREAD_LEVEL; threads = 2^Level.

Threads (cores)	Level	Sequential mergesort (s), mean ± sd	Parallel mergesort (s), mean ± sd	Speedup	Linear	Efficiency
1	0	2.516 ± 0.007	2.522 ± 0.007	1.00	1	100%
2	1	2.518 ± 0.005	1.315 ± 0.005	1.91	2	96%
4	2	2.523 ± 0.007	0.742 ± 0.001	3.40	4	85%
8	3	2.521 ± 0.005	0.470 ± 0.003	5.37	8	67%
16	4	2.516 ± 0.009	0.383 ± 0.019	6.57	16	41%

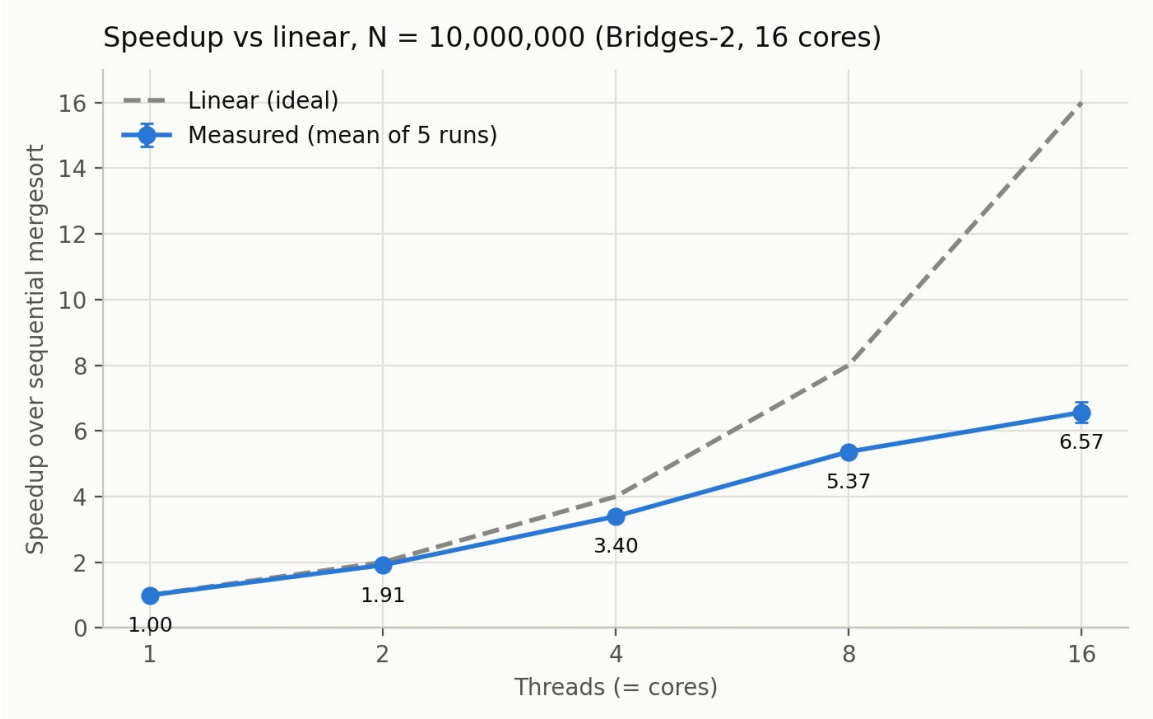


Figure 1. Measured speedup compared with linear speedup. Error bars are the standard deviation of the per-run speedup.

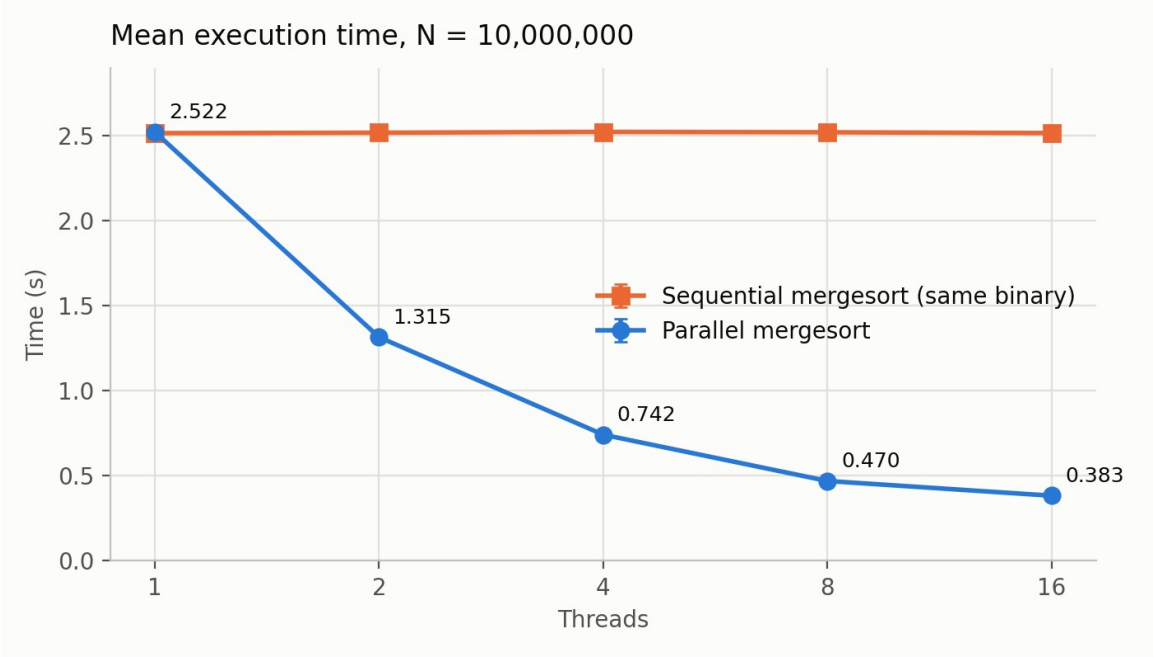


Figure 2. Mean execution time. The sequential mergesort line is flat because every executable runs the same sequential code.

Discussion. With 2 threads the speedup is 1.91 (96% efficiency), almost linear. Efficiency then drops steadily: 85% at 4, 67% at 8 and 41% at 16 threads, where the speedup is 6.57 instead of 16. The 1-thread version is a useful check: it runs the same sequential code inside one extra thread and takes the same time as the sequential mergesort (speedup 1.00), so thread creation itself costs almost nothing. The gap from linear comes from the merges, which Task 2 measures.

The runs were very repeatable. The standard deviation is under 1% of the mean everywhere except 16 threads (0.019 s, 5%). Node r195 is shared with other users' jobs (Slurm reported a CPU load of about 111 of its 128 cores while this job ran), so with 16 threads my job competes for memory bandwidth and shared L3 cache with them.

Raw output (one complete run per configuration, copied verbatim from pa3-50358707.out)

```
host: r195.ib.bridges2.psc.edu  cpus: 16  date: Mon Nov  9 01:05:16 EST 2026
=== threads=1 run=1
Top-level sequential merge took: 0.106852 sec.
Sequential mergesort took: 2.510906 sec.
Parallel mergesort took: 2.525512 sec.
Built-in qsort took: 2.182189 sec.
=== threads=2 run=1
Top-level sequential merge took: 0.106794 sec.
Sequential mergesort took: 2.519930 sec.
Top-level merge in parallel run took: 0.105430 sec.
Parallel mergesort took: 1.313357 sec.
Built-in qsort took: 2.310365 sec.
=== threads=4 run=1
Top-level sequential merge took: 0.106050 sec.
Sequential mergesort took: 2.515865 sec.
Top-level merge in parallel run took: 0.107519 sec.
Parallel mergesort took: 0.740828 sec.
Built-in qsort took: 2.311380 sec.
=== threads=8 run=1
Top-level sequential merge took: 0.108341 sec.
Sequential mergesort took: 2.528775 sec.
```

```
Top-level merge in parallel run took: 0.107768 sec.
Parallel mergesort took: 0.473903 sec.
Built-in qsort took: 2.282041 sec.
=== threads=16 run=1
Top-level sequential merge took: 0.107090 sec.
Sequential mergesort took: 2.509709 sec.
Top-level merge in parallel run took: 0.107153 sec.
Parallel mergesort took: 0.386546 sec.
Built-in qsort took: 2.311228 sec.
```

Evidence that the threads really run in parallel (job 50358766)

A third job on the same kind of 16-core allocation (node r227, CPUs 49–63 and 127) collected direct evidence of what the threads do.

Thread snapshot. I started `mergesort16` on 100,000,000 elements and ran `ps -L` once it had created its threads (after 30s, 32 threads in process 9520). The PSR column is the CPU each thread is on, and STAT R means running, S sleeping:

```
  PID    LWP PSR %CPU STAT COMMAND
 9520   9520  54 98.3 Sl   ev16
 9520   9642  57  0.0 Sl   ev16
 9520   9643  63  0.0 Sl   ev16
 9520   9644  54  0.0 Sl   ev16
 9520   9645  57  0.0 Sl   ev16
 9520   9646  52  0.0 Sl   ev16
 9520   9647  60  0.0 Sl   ev16
 9520   9648  61  0.0 Sl   ev16
 9520   9649  62  0.0 Sl   ev16
 9520   9650  63  0.0 Sl   ev16
 9520   9651  51  0.0 Sl   ev16
 9520   9652  58  0.0 Sl   ev16
 9520   9653  57  0.0 Rl   ev16
 9520   9654  54  0.0 Sl   ev16
 9520   9655  58  0.0 Sl   ev16
 9520   9656  53  0.0 Sl   ev16
 9520   9657  56  0.0 Rl   ev16
 9520   9658  52  0.0 Rl   ev16
 9520   9659  50  0.0 Sl   ev16
 9520   9660  62  0.0 Rl   ev16
 9520   9661  63  0.0 Rl   ev16
 9520   9662  59  0.0 Rl   ev16
 9520   9663  50  0.0 Rl   ev16
 9520   9664  51  0.0 Rl   ev16
 9520   9665  55  0.0 Rl   ev16
 9520   9666  49  0.0 Rl   ev16
 9520   9667 127  0.0 Rl   ev16
 9520   9669  53  0.0 Rl   ev16
 9520   9668  61  0.0 Rl   ev16
 9520   9670  60  0.0 Rl   ev16
 9520   9671  58  0.0 Rl   ev16
 9520   9672  54  0.0 Rl   ev16
```

There are 32 threads: the main thread plus 2^5 − 1 = 31 created by `parallelMergeSortHelper` with THREAD_LEVEL 4. **16 are running, each on a different CPU** (16 distinct CPUs, exactly the 16 Slurm gave the job). These are the 16 leaf threads sorting N/16 elements each. The other 16 are sleeping: the main thread and the 15 internal threads, blocked in `pthread_join` until their children finish, after which each one does its merge. These are the fork and merge nodes of the DAG in Task 3.

CPU time. `/usr/bin/time -v` on the whole program (sequential sort, parallel sort and qsort) for 1 and 16 threads, N = 10,000,000:

Table 2. /usr/bin/time -v, whole program (verbatim values).

Threads	User CPU time (s)	Wall-clock time	Percent of CPU this job got	Parallel mergesort (s)
1	7.60	0:07.71	99%	2.660949 sec.
16	7.58	0:05.52	138%	0.452649 sec.

The total CPU time is the same (7.60 s vs 7.58 s), so the parallel version does the same work and only spreads it over more cores; this is the work law from Task 3 (W = T1). The wall-clock time drops by about the time the parallel sort saved. The CPU percentage for the whole program is only 138% because the sequential mergesort and qsort, which take most of the run, use one core. This is also why Slurm's `seff` reports low CPU efficiency for these jobs.

Hardware counters. `perf stat` over the whole program:

Table 3. perf stat, whole program, N = 10,000,000 (verbatim counts).

Threads	Instructions	Cycles	Instructions per cycle	Cache-miss rate
1	30141054862	24476538533	1.23	5.084%
16	30113531736	24604566817	1.22	4.976%

The instruction counts are the same to within 0.1%, which again confirms that the work does not change. The instructions per cycle and the cache-miss rate are also the same, but that does not mean the parallel part has no memory contention: `perf` counted the whole program, and more than 90% of it is the single-threaded sequential sort and qsort. The per-level merge profile at the end of Task 2 isolates the parallel merges and does show them slowing down.

Input size. Task 3 predicts that the parallelism grows only like (lg N)/2, so a bigger input should help a little. I ran 1 and 16 threads on three sizes, 3 runs each:

Table 4. Input-size sweep at 16 threads (mean of 3 runs, job 50358766).

N	Sequential (s)	Parallel, 16 threads (s)	Speedup	f = 2Tm/T1	Amdahl S(16)	(lg N)/2
1,000,000	0.226	0.046 ± 0.007	4.92	0.1076	6.12	10.0
10,000,000	2.637	0.420 ± 0.046	6.28	0.0856	7.00	11.6
100,000,000	28.577	4.911 ± 0.077	5.82	0.0737	7.60	13.3

The predicted speedup grows slowly with N, from 6.12 to 7.60. The measurements only partly follow it. At 1,000,000 elements the parallel sort takes only 0.046 s, so creating 31 threads and the uneven start of the leaf threads cost a larger share, and the speedup is lowest (4.92). At 100,000,000 the results vary a lot: the three runs averaged 5.82, but the run sampled by `ps` in the same job got 7.82. With two 800 MB arrays plus the 800 MB backup copy, this size depends heavily on memory bandwidth, which the node shares with other jobs. The cleaner comparison is the 50,000,000-element run in Table 7, where 16 threads gave 7.46 compared with 6.57 at 10,000,000, as predicted.

Task 2. (15 pts) Timing the sequential merge and Amdahl's Law

Timing code

In the provided code the sequential mergesort is `mergeSort()` → `mergeSortHelper(lyst, back, 0, size-1)`, and the top-level merge is the last `merge()` call inside the first `mergeSortHelper` call. To time only that call, I unrolled the first level of the recursion in `mergeSort()` so the top-level `merge()` is called directly and wrapped it in `gettimeofday()` calls (`parallelMergesort.c`, lines 153–171). The two halves are split at the same `mid` that `mergeSortHelper` uses, so the work is unchanged.

```
void mergeSort(double lyst[], int size)
{
    struct timeval start, end;
    double *back = (double *) malloc(size*sizeof(double));

    if (size > 1)
    {
        int mid = (size - 1) / 2;
        mergeSortHelper(lyst, back, 0, mid);
        mergeSortHelper(lyst, back, mid + 1, size - 1);

        gettimeofday(&start, NULL);
        merge(lyst, back, 0, mid, size - 1);
        gettimeofday(&end, NULL);
        printf("Top-level sequential merge took: %lf sec.\n",
            elapsed(&start, &end));
    }
    free(back);
}
```

I also timed the same top-level merge inside the parallel version (`parallelMergeSortHelper`, lines 360–370). It prints only for the thread whose range is the whole list, which is the merge done after the last `pthread_join`. `elapsed()` is a small helper with the same microsecond arithmetic `main()` already used. I also added a usage message: the program now exits with status 1 if it is given more than one argument or a size below 2, instead of crashing.

```
    struct timeval start, end;
    gettimeofday(&start, NULL);
    merge(my_data->lyst, my_data->back, my_data->low, mid, my_data->high);
    gettimeofday(&end, NULL);
    if (my_data->low == 0 && my_data->high == NUM - 1)
    {
        printf("Top-level merge in parallel run took: %lf sec.\n",
            elapsed(&start, &end));
    }
```

Measured merge time

Table 5. Top-level merge times (mean ± sd over 5 runs, N = 10,000,000).

Threads	Top-level merge, sequential sort (s)	Top-level merge, parallel sort (s)	Merge share of sequential time
1	0.1074 ± 0.0015	– (no merge: one thread sorts everything)	4.27%
2	0.1072 ± 0.0009	0.1071 ± 0.0017	4.26%
4	0.1068 ± 0.0006	0.1077 ± 0.0010	4.23%
8	0.1072 ± 0.0011	0.1073 ± 0.0004	4.25%
16	0.1065 ± 0.0004	0.1081 ± 0.0024	4.23%

Averaged over all 25 runs, the top-level sequential merge takes **Tm = 0.1070 s** and the whole sequential mergesort takes **T1 = 2.5187 s**. The top merge is the same in the parallel version (0.1081 s at 16 threads), which confirms it is done by a single thread there too.

A useful cross-check: every level of the merge tree merges all N elements once, so each level should cost about Tm. A full mergesort has log2(10,000,000) = 23.25 levels, which predicts 23.25 × 0.1070 = 2.489 s for the whole sequential sort. The measured 2.519 s is within 1.2%, so the per-level merge cost really is constant and Tm is a good unit of work.

Why the measured merge time needs more calculation

Only the top-level merge is completely sequential, but it is not the only merge that limits the parallel execution. In `parallelMergeSortHelper` a thread at level k waits for its two children, then merges their halves itself. So with p = 2^L threads the merge tree has L levels that run with less than full parallelism:

- level 0 (top): 1 merge of N elements, 1 thread busy → takes Tm
- level 1: 2 merges of N/2 elements at the same time → takes Tm/2
- level 2: 4 merges of N/4 → Tm/4, … , level L−1: 2^(L−1) merges → Tm/2^(L−1)

While these merges run, most of the p cores are idle, so their time adds directly to the parallel execution time. The total is a geometric series:

Ts(L) = Tm (1 + 1/2 + 1/4 + ... + 1/2^(L−1)) = Tm (2 − 2^(1−L)), which approaches 2·Tm as L grows.

So the serial time in the parallel execution is about twice the measured top-level merge time, not Tm. This is the extra calculation the handout asks for. For an unlimited number of cores, Ts = 2·Tm = 0.2140 s.

Amdahl's Law predictions

Serial fraction f = 2·Tm / T1 = 0.2140 / 2.5187 = **0.0850**. For comparison, using only the top merge would give f = Tm / T1 = 0.0425.

Amdahl's Law: S(p) = 1 / (f + (1 − f)/p). With infinitely many cores, **S(∞) = 1/f = 11.8**. With the 16 cores I used, **S(16) = 7.03**. (Using the top merge only would predict S(∞) = 23.5 and S(16) = 9.77, which is far above what I measured.)

Table 6. Predicted and measured speedup. Measurements for 32 and 64 threads come from the second job (64-core allocation); see below.

Threads p	Measured speedup	Amdahl, f = 0.085	Amdahl, f = 0.042 (top merge only)	Work-span model T1 / span	Linear
1	1.00	1.00	1.00	1.01	1
2	1.91	1.84	1.92	1.94	2
4	3.40	3.19	3.55	3.45	4
8	5.37	5.02	6.17	5.50	8
16	6.57	7.03	9.77	7.64	16
32	7.68	8.80	13.81	9.38	32
64	7.91	10.07	17.41	10.52	64
∞	–	11.77	23.54	≈ 11.8	∞

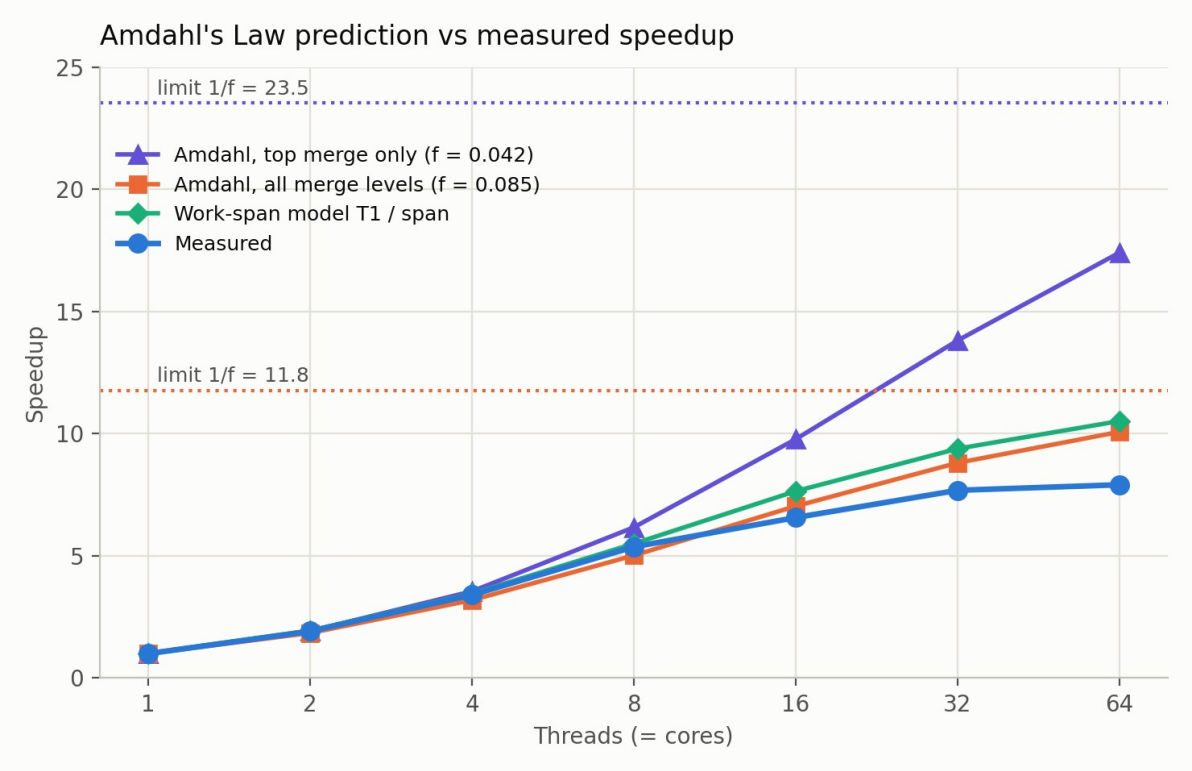


Figure 3. Amdahl's Law predictions vs measured speedup. The dotted lines are the 1/f limits.

Discussion

Using all merge levels is the right serial fraction. With f = 0.085, Amdahl predicts 7.03 at 16 threads and I measured 6.57 (93% of the prediction). Counting only the top merge predicts 9.77, which overestimates by 49%.

At 2–8 threads the measurements are slightly above the f = 0.085 curve. Amdahl's Law assumes the serial part is fixed, but here it grows with the number of threads: with 2 threads only the top merge is serial (Tm), with 4 threads it is 1.5·Tm, and it reaches 2·Tm only in the limit. The work-span model in

Table 6 uses the exact serial part for each L, and it matches the measurements within 2% for 2, 4 and 8 threads.

At 16 threads the measurement falls below both models (0.383 s measured vs 0.329 s predicted by the work-span model). Neither model includes hardware effects. Merging is memory-bound: it reads and writes every element with almost no computation, so 16 threads sorting and merging at once compete for memory bandwidth and the shared L3 cache. The node was also shared with other users' jobs, and the 16-thread runs were the only ones with noticeable variation. Both models also assume the merges at each level run perfectly in parallel; the per-level merge profile below shows they are slower than that.

Extra experiments (job 50358714)

To check the conclusions above, a second job (50358714) ran on a 64-core RM-shared allocation (node r358, same AMD EPYC 7742 hardware). It ran four experiments; every run passed the same validation as the main job.

More threads and a bigger input. With N = 10,000,000 the speedup is 7.68 at 32 threads and 7.91 at 64, so it has nearly stopped growing, as Amdahl's Law predicts (limit 11.8). With N = 50,000,000 (sequential mergesort 14.08 s, top-level merge 0.5467 s, so f = 0.0777), Amdahl predicts S(16) = 7.39 and S(64) = 10.86; I measured 7.46 and 9.37. The larger input has a smaller serial fraction because the merges are Θ(N) while the whole sort is Θ(N log N), so it scales a little further. This is the Θ(log N) parallelism from Task 3.

Table 7. Speedup for the extra experiments (mean of 5 runs for A, 3 runs for B and D).

Threads	N = 10M, -O0	N = 50M, -O0	Amdahl N = 50M (f = 0.078)	N = 10M, -O2
1	1.00	1.00	1.00	0.99
2	1.91	1.90	1.86	1.90
4	3.40	3.43	3.24	3.32
8	5.37	5.40	5.18	5.19
16	6.57	7.46	7.39	6.97
32	7.68	9.01	9.39	8.45
64	7.91	9.37	10.86	8.65

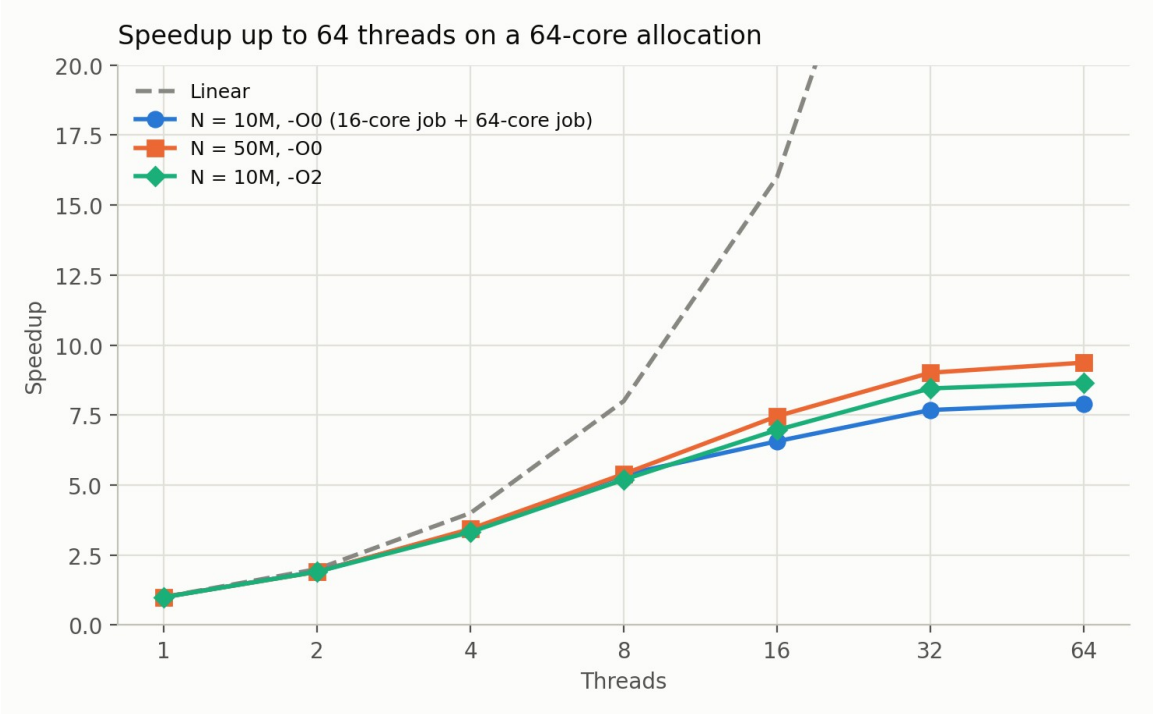


Figure 4. Speedup up to 64 threads. The 1–16 thread points for N = 10M are from the main job; 32 and 64 are from the 64-core job.

Compiler optimization. With `-O2` the sequential mergesort takes 1.632 s instead of 2.519 s and the top-level merge 0.0690 s instead of 0.1070 s. Both get about 1.54× faster, so the serial fraction barely changes (2·Tm/T1 = 0.0845) and neither does the speedup (6.97 at 16 threads). Optimization makes the program faster but not more parallel.

Per-level merge times. Building with `-DMERGE_PROFILE` makes every thread print how long its merge took. For 64 threads (THREAD_LEVEL 6) that is 63 merges per run on six levels. Table 8 compares them with the model from Task 2, which assumes a merge of N/2^k elements takes Tm/2^k.

Table 8. Merge time per level, 64 threads, 3 runs (ideal uses Tm = 0.1094 s measured in the same runs).

Level k	Merge size	Merges per run	Measured mean (s)	Ideal Tm/2^k (s)	Slower by
0	N/1	1	0.1286	0.1094	17%
1	N/2	2	0.0670	0.0547	22%
2	N/4	4	0.0343	0.0274	26%
3	N/8	8	0.0180	0.0137	32%
4	N/16	16	0.0091	0.0068	33%
5	N/32	32	0.0045	0.0034	31%

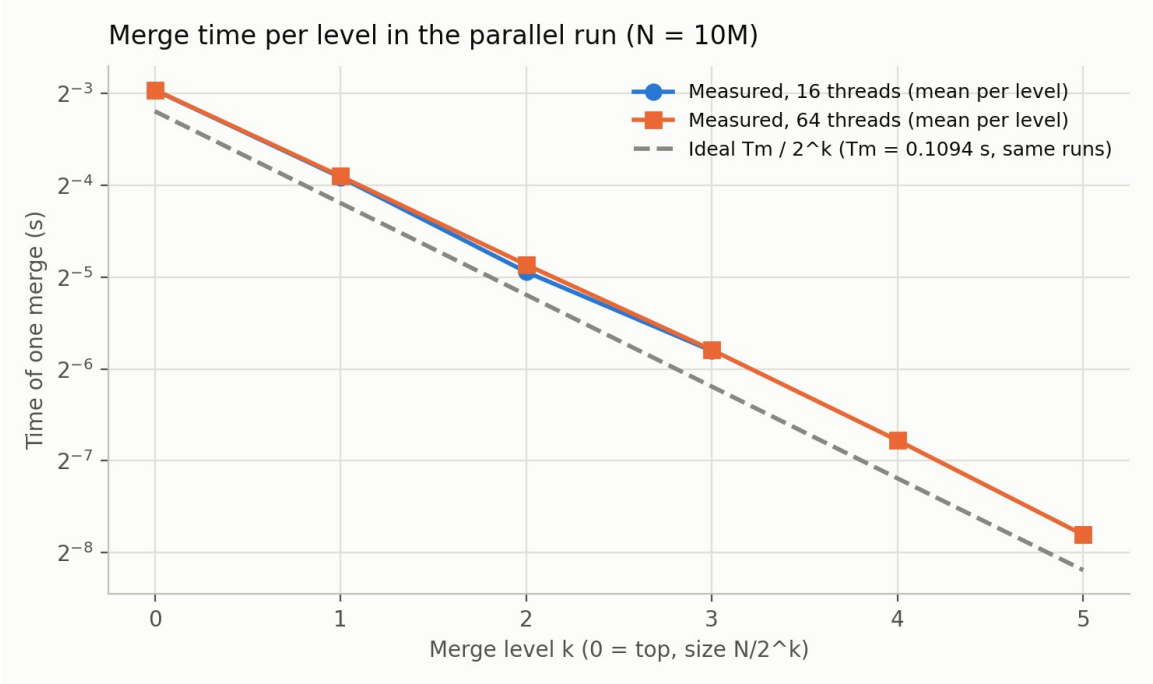


Figure 5. Time of one merge at each level of the parallel run (log scale).

Every level is slower than the model, by about 17% at the top and 22–33% below it. Two effects explain this. The merges on the same level run at the same time and share the memory bandwidth. And each merge reads data that other threads just wrote on other cores, so it misses in its own cache. This is the main reason the measured speedup falls below both models at 16 threads and above.

Task 3. (15 pts) Computational DAG and work-span analysis

Figure 6 is the DAG for `THREAD_LEVEL = 3` (8 threads). Each node is a piece of sequential work labeled with its cost in element operations. Edges are dependencies: a fork node creates two threads (`pthread_create`); a merge node can start only after both child threads finish (`pthread_join`). In the code, the fork and merge nodes at the same position belong to the same thread, which sleeps in `pthread_join` between them.

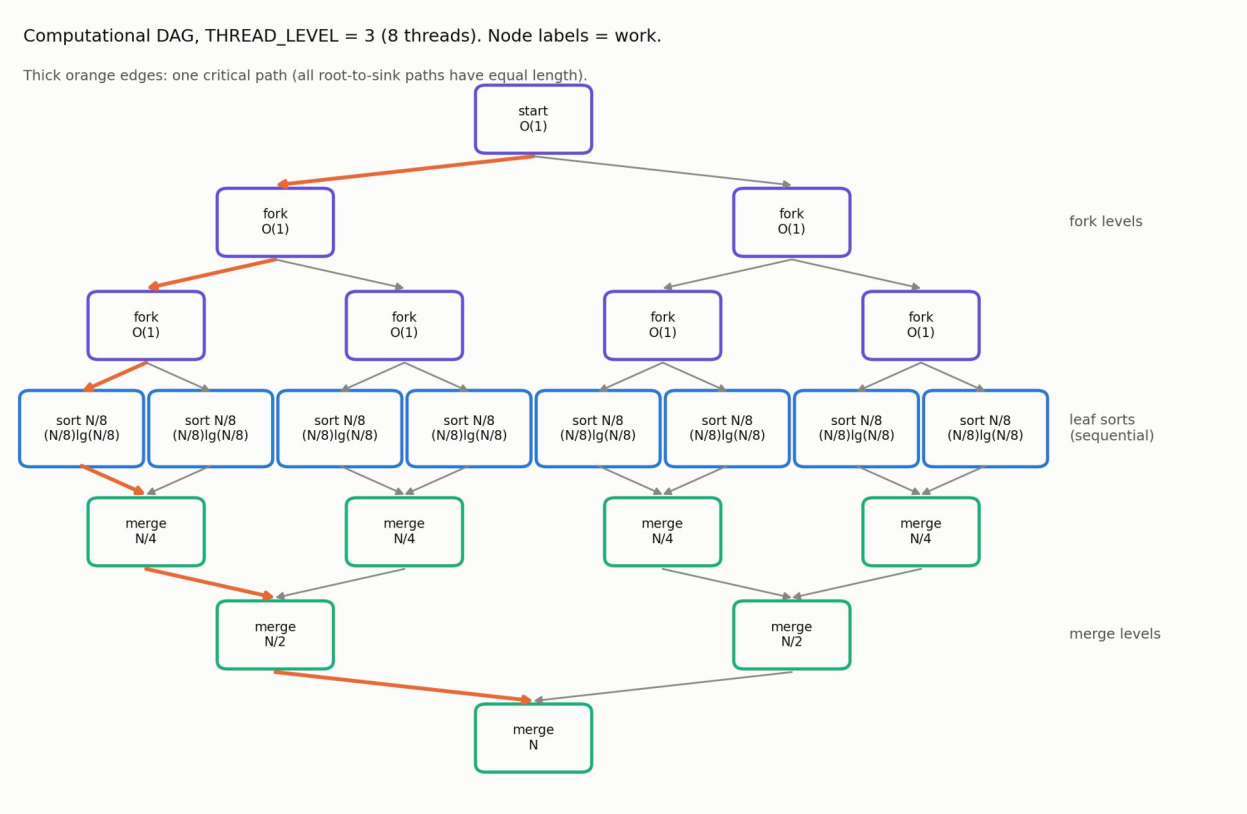


Figure 6. DAG for 8 threads. Purple: thread creation, O(1). Blue: sequential mergesort of N/8 elements, c·(N/8)·lg(N/8). Green: merges with their sizes (a merge of n elements costs c·n). Orange: one critical path.

Work of each node (c = cost per element per merge level; Task 2 measured c ≈ Tm/N = 10.7 ns):

- fork / start node: O(1) (one `pthread_create` pair). There are p − 1 of them.
- leaf node: `mergeSortHelper` on N/p elements, c·(N/p)·lg(N/p). There are p of them.
- merge node at level k (k = 0 is the top): merges N/2^k elements, c·N/2^k. Level k has 2^k of them.

The DAG has logarithmic depth (L fork levels and L merge levels), but its nodes do not have constant work: the leaf and merge nodes are large, and the merge nodes get bigger toward the bottom. So the span has to be found by adding the node weights along a path, not by counting levels.

Work

$$W(N, p) = p \cdot c(N/p)\lg(N/p) + L \cdot cN + O(p) = cN(\lg N - L) + cN{\cdot}L + O(p) = cN \lg N + O(p)$$

Each merge level touches all N elements once, so L levels of merges cost cN·L, and the leaves supply the remaining lg N − L levels. The work equals the sequential mergesort, Θ(N log N), so parallelizing adds no extra work apart from thread creation. Measured: W ≈ T1 = 2.519 s.

Span (critical path)

Every root-to-sink path goes through L forks, one leaf and one merge on each level, and all of these have the same weight, so every path is critical. The orange path in Figure 6 is one of them. Adding the node weights:

$$S(N, p) = O(L) + c(N/p)\lg(N/p) + c(N + N/2 + \ldots + N/2^{(L-1)}) = c(N/p)\lg(N/p) + 2cN(1 - 2^{(-L)}) + O(L)$$

This is the same geometric series as in Task 2: the merges on the critical path add up to almost 2N element operations no matter how many threads are used.

Table 9. Work-span numbers using the measured c (from Tm). W = T1 = 2.519 s.

Threads p	Leaf on path (s)	Merges on path (s)	Span S (s)	W / p (s)	Lower bound max(W/p, S) (s)	Measured Tp (s)	Parallelism W / S
2	1.191	0.107	1.298	1.259	1.298	1.315	1.94
4	0.569	0.161	0.729	0.630	0.729	0.742	3.45
8	0.271	0.187	0.458	0.315	0.458	0.470	5.50
16	0.129	0.201	0.329	0.157	0.329	0.383	7.65
32	0.061	0.207	0.268	0.079	0.268	0.337	9.38
64	0.029	0.211	0.240	0.039	0.240	0.327	10.51

Lower bounds. The work law gives Tp ≥ W/p and the span law gives Tp ≥ S. For every thread count the span is the larger of the two, so the span law is the bound that matters, and every measured time is above it, as it must be. For 2, 4 and 8 threads the measured time is within 3% of the span, which means the program is nearly as fast as this DAG allows.

Unlimited threads. If the recursion created threads all the way down (L = lg N), the leaves would vanish and S = 2cN(1 − 1/N) + O(lg N) ≈ 2cN = Θ(N). The parallelism would be W/S = cN lg N / 2cN = (lg N)/2 ≈ 11.6 for N = 10^7. This agrees with the Amdahl limit of 11.8 from Task 2, which is expected because both come from the same 2·Tm of merges.

Implications of the critical path. The critical path is dominated by the merges, and above all by the final merge of all N elements, which one thread does alone. No number of cores can make the program faster than about 2·Tm ≈ 0.214 s, and the parallelism only grows like Θ(log N), so larger inputs barely help. To do better, the merge itself has to be parallelized. Grossman section 5.4 shows how: split the larger half at its median, binary-search for that value in the other half, and merge the two pairs of pieces in parallel. That merge has span $O(\log^2 N)$, so the whole sort has span $O(\log^3 N)$ and parallelism $\Theta(N / \log^2 N)$, which is enough to keep many more cores busy.

Task 4. (10 pts) Difficulties and what I learned

[Draft written by Claude from what actually happened while doing this assignment. Gael: rewrite this section in your own words before submitting.]

Difficulties

- **Getting the code off Bridges-2.** My first `scp` closed the connection right after I entered my password. Logging in interactively with `ssh` worked. The next `scp` failed with "subsystem request failed", because newer macOS `scp` uses SFTP and Bridges-2 login nodes don't allow it. `scp -O` (the old protocol) then failed with "scp: command not found", because the login node has no `scp` program. What finally worked was streaming a tar archive over plain ssh: `ssh sustaita@bridges2.psc.edu 'tar cf - ...' | tar xf -`.
- **Typing my password for every command.** I opened one SSH connection with `ControlMaster/ControlPersist` so later `ssh` commands reused it without asking for the password again. That made it practical to upload files, submit jobs and fetch results from scripts.
- **Where to put the timer.** The top-level sequential merge is buried inside the recursive `mergeSortHelper`, so there was no single call to wrap. I unrolled the first level of the recursion in `mergeSort()` so the top-level `merge()` is a separate call that can be timed, without changing the work done.
- **Why the merge time needs more calculation.** At first I used only the top-level merge as the serial part, and Amdahl's Law predicted almost 10× at 16 threads while I measured 6.6×. Drawing the DAG showed that the lower merge levels are also only partly parallel, and summing them gives about 2·Tm. With that, the prediction matches.
- **Run time and noise.** With N = 10,000,000 the sequential sort takes only about 2.5 s, and the node was shared with other jobs. I ran every configuration 5 times, reported the standard deviation, and ran a second job with N = 50,000,000 to check that the conclusions hold with longer runs.
- **Results that did not match the model.** The input-size sweep did not increase steadily: 16 threads gave 4.92× on 1,000,000 elements, 6.28× on 10,000,000 and 5.82× on 100,000,000, and one 100,000,000-element run in the same job got 7.82×. I had to work out that very large inputs depend on memory bandwidth shared with other jobs, and use the cleaner 50,000,000-element runs to check the trend. `perf stat` also showed no difference between 1 and 16 threads, until I realized it was counting the whole program, which is mostly single-threaded.
- **The handout's default THREAD_LEVEL.** The handout says the default is 2^3, but the shared file had `THREAD_LEVEL 1`. My build script sets it explicitly and checks that the change was made.

What I learned

- How to measure speedup properly: compare against the sequential version on the same input, repeat runs, and look at the spread, not one number.
- Amdahl's Law is only as good as the serial fraction you put into it. The serial part of a divide-and-conquer program is the whole chain of partly-parallel combine steps, not just the last one.
- Work and span explain the results: the work of parallel mergesort is the same Θ(N log N) as sequential, but the span is Θ(N) because of the sequential merge, so the parallelism is only about (log N)/2 ≈ 12. Adding cores past that point cannot help.

- Real hardware falls below the models. Memory bandwidth, shared caches and other users on the node cost speedup at 16 threads and more, which neither Amdahl's Law nor work-span accounts for.
- How to use a batch system (Slurm `sbatch`, `squeue`, `sacct`) on a supercomputer, and how to move files when the usual tools are blocked.

Resources and AI-use disclosure

[Gael: review and edit this disclosure so it accurately describes your own use before submitting.]

Resources: the PA3 handout; the provided parallelMergesort.c (Joshua Stough, MIT License); Grossman, “A Sophomoric Introduction to Shared-Memory Parallelism and Concurrency”, section 5.4; the PSC Bridges-2 User Guide.

AI use: I used Claude Code (Anthropic, model Claude Opus 5.5) as an assistant throughout this assignment. Claude explained how to log in to Bridges-2 and how to work around the blocked scp/SFTP, and wrote a step-by-step page of the login and job-submission commands. After I opened an SSH connection with my own password, Claude used it to upload the code, submit the Slurm jobs and download the output. Claude wrote the timing code in parallelMergesort.c (the unrolled top-level merge timer, the parallel top-level merge timer, the optional MERGE_PROFILE output and the usage check), the batch scripts run.sbatch, run_extra.sbatch and run_demo.sbatch, the demo script demo.py, the Python scripts that validate the output and compute the statistics, Amdahl's Law and work-span numbers, all figures and the spreadsheet, and drafted the text of this report, including the Task 4 reflection, which I then rewrote. All numbers in this report come from the Bridges-2 runs listed above; none were typed in by hand. I reviewed the code, results and analysis and am responsible for the submitted work.

Appendix: files in the repository

File	What it is
parallelMergesort.c	Provided program plus the Task 2 timers, the elapsed() helper, a usage check and optional MERGE_PROFILE output.
run.sbatch	Main Slurm job: builds the 1/2/4/8/16-thread executables and runs each 5 times on 10,000,000 elements (16 cores).
run_extra.sbatch	Extra job on 64 cores: 32/64 threads, N = 50,000,000, per-level merge profile, -O2 builds, and the node's hardware info.
run_demo.sbatch	Evidence job on 16 cores: /usr/bin/time -v, a ps -L thread snapshot, perf stat, and the input-size sweep.
results/pa3-50358707.out, pa3-extra-50358714.out, pa3-evidence-50358766.out	Raw Bridges-2 output of the three jobs, unedited.
results/sacct.txt	Slurm accounting (sacct, seff) for the three jobs: all COMPLETED with exit code 0.
demo.py	Demo script: builds and runs the 1–16 thread versions locally, prints the timer code, and shows the Bridges-2 results task by task.
results/runs.csv, summary.csv, merge_profile.csv	Every run and the per-configuration statistics, produced by parse.py after validation.
results/amdahl.csv, constants.csv	Amdahl and work-span predictions, produced by analyze.py.
results/PA3 results.xlsx	Spreadsheet: raw runs, a summary computed with formulas, and an Amdahl sheet where T1 and Tm can be changed to recompute every prediction.
results/figs/	All figures in this report.
analysis/parse.py, analysis/analyze.py	Validation, statistics and figure scripts.